# Jeopardy! as a Question Answering Challenge: A Hybrid Sparse-Dense Retrieval Approach over Wikipedia

Aunabil Chakma, Cole Mobberly, Suhani Surana
University of Arizona

**Abstract**

We present JEOPARDY_PROGRAM which is a Q and A system that retrieved Wikipedia articles when provided with the questions. We rank approximately 280,000 Wikipedia pages and return the top 10, top 5 and top 1 likely answer title for each question. Our system combines BM25 sparse retrieval with Dense Passage Retrieval (DPR)in a weighted hybrid architecture.

NEED TO FILL THIS.

## 1 Introduction

The development of open-domain Question Answering (QA) systems has been a central challenge in natural language processing and information retrieval. IBM Watson demonstrated the ability to compete against humans on Jeopardy!. Such systems must interpret complex natural language clues, retrieve relevant information from vast corpora, and identify precise answers in real time. While the full pipeline involves sophisticated reasoning and evidence synthesis, a significant subset of QA tasks can be reframed as an information retrieval (IR) problem.

In particular, many Jeopardy! clues have answers that are actually titles to the Wikipedia pages. This observation allows us to simplify the QA task into one of document retrieval which is, given a natural language clue, the goal is to identify the Wikipedia page whose title best matches the intended answer. This formulation shifts the focus from deep semantic reasoning to effective indexing, query construction, and similarity matching within a large textual corpus.

In this project, we explore this retrieval-based approach using a dataset consisting of 100 Jeopardy! clues paired with answers that appear as Wikipedia page titles, alongside a large corpus of approximately 280,000 Wikipedia articles. We implement an IR system to index these documents and design a retrieval pipeline that, given a clue, ranks candidate pages based on their relevance. Key challenges include preprocessing noisy and heterogeneous Wikipedia text, selecting informative query terms from often verbose or indirect clues, and optimizing retrieval performance.

By systematically analyzing indexing strategies, term processing techniques, and query formulation methods, this work aims to evaluate how effectively a classical IR framework can approximate the behavior of a QA system in a constrained but realistic setting.

## 2 Core Implementation: Indexing and Retrieval

### 2.1 Wikipedia Preprocessing

We first start with a Wikipedia collection of approximately 280,000 articles. These articles are distributed across 80 raw files. These contain a sequence of multiple articles and each page is separated by a [[Title]]

in this format. A preprocessing pipeline parses these files and stores each article as a separate record in a SQLite database, recording the title, raw body text, source file identifier, article index within that file, and a boolean flag indicating whether the article is a redirect. This separation by articles gives us an advantage since when we index each wiki page as a separate document rather than concatenating all the documents into a single document, it allows our retrieval system to return a specific page title as the answer.

Then, we strip the redirect lines, section headers, wiki markup templates, reference annotations, file and image markup, and category lines, and then normalize whitespace. Wikipedia pages contain a lot of dense markup and if we index it in the raw form, we introduce several non informative and high frequency tokens that would artificially inflate the term frequencies and distort BM25 scores. We also identify redirect articles during parsing and route them to redirect index since including such articles' body test in the main index would include noise as they include no significant original content and instead serve only as a redirect to the primary page. We describe this in detail, in Section 2.2.

All the clean and non redirect articles are stored in a SQLite database. This will now serve as a primary source for all the further processed indexes.

## 2.2 Sparse Index Construction

For sparse indexing, we use the Whoosh library which is a a fast, full-text indexing and searching library and it implements BM25F scoring.

We divide sparse indexing into two parts, for non redirect articles and redirect articles. For non-redirect articles, we index it under two fields which are `title` field containing the cleaned article title, and a `body` field containing the full cleaned article text.

We use Whoosh's default analyzer for spare index construction. It applies a standard tokenizer, a lowercase filter, and a stemming filter based on the Porter stemmer which reduces morphological variants to a common root form. To keep common filler words from skewing frequency data, Whoosh automatically filters out stop words using Whoosh's built-in stop word list.

We do not apply any additional lemmatization other than Porter stemming since it is cuffiecient No additional lemmatization is applied beyond Porter stemming, since the Porter stemmer is robust enough to handle the typical word variations found in both Wikipedia articles and Jeopardy clues.

We did this by indexing not just the full articles, but also three lead-text indexes; one for the first sentence, one for the first two sentences, and one for the first paragraph. This works well because Wikipedia articles usually start with the most important information ie the information in concentrated in the first paragraph. When we analyzed the 100 questions, majority of the questions had a single fact behind them and typically corresponded to content appearing in the title or the first sentence or first paragraph of the target Wikipedia article. Thus, by providing these dedicated indexes over these lead variants allows the retrieval system to assign higher weights to documents whose most prominent content matches the clue without having to go through the whole main body which requires computational and storage resources and would go through unrelated terms.

We handle redirect articles through a dedicated redirect index and a canonical resolution database. When a redirect article gets a high BM25 score for a query, we transfer that score to its final target article. We use a precomputed lookup table to resolve each redirect chain to its canonical page. This mechanism ensures that evidence from redirect titles adds to the score of the correct article rather than competing with it in the final ranking, and that our retrieval system never returns a redirect page title as its predicted answer.

A final sparse index stores semantic category tags derived from the Jeopardy! category field. Each unique Jeopardy! category is embedded using the `all-MiniLM-L6-v2` sentence encoder (**?**), and each article is represented as the concatenation of its title with its first sentence and, separately, with its first two sentences. The top-10 most semantically similar category names for each article representation are stored as a keyword field in a dedicated category index. At retrieval time, the category text of the incoming question is matched

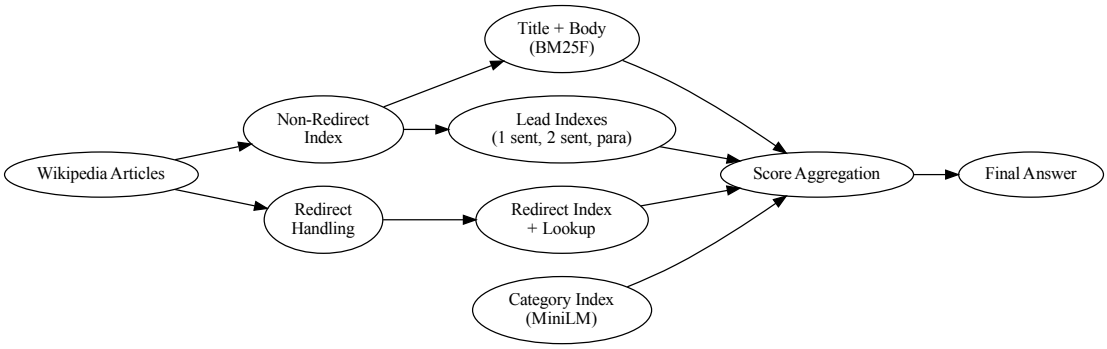


Figure 1: Enter Caption

against these keyword fields, providing a domain-restriction signal that is particularly useful for clues in categories such as NEWSPAPERS, BROADWAY LYRICS, or AFRICAN CITIES, where the category alone substantially narrows the plausible answer space.

## 2.3 Dense Index Construction

To complement sparse lexical matching, we also build a dense retrieval component based on the Dense Passage Retrieval (DPR) framework. So, each article (which is non directed) is represented in four variants: (1) title + first sentence, (2) title + first two sentences, (3) title + first paragraph, and (4) title + full article body. This design allows the model to capture both highly focused and more complete contextual information.

Each (title, passage variant) pair is encoded using the facebook/dpr-ctx_encoder-single-nq-base model. Then, the resulting dense vectors are stored in a FAISS `IndexFlatIP`. This index performs exact nearest-neighbor search using inner product similarity.

We thus maintain four separate FAISS indexes (according to the required passage granularity). At query time, the system searches across all four indexes and lets the retrieval system match queries against different levels of article detail. It is not restricted to a single fixed passage length. Instead, it can retrieve documents based on whichever representation best captures the defining information.

## 2.4 Query Construction

When we analysed the 100 clues, we identified several structural patterns. 61/100 clues had clear multi word, title-cased phrases. These usually refer to names of people, places, or works which act as strong anchors for retrieval. 41/100 clues contained atleast one four digit year. 53/100 clues contained quoted text (most of them from song lyrics or book text or newspaper articles etc.). These often match article lead text for first paragraph of the body. 57/100 clues used ”this” instead of the answer directly so the answer would have to be retrieved on the adjacent words to inform the system about the context.

Thus, based on this analysis, we made it support two query construction modes. In *full* mode, the complete clue text is submitted to the retrieval components unchanged, preserving all terms including function words. In *entity* mode, the query is reduced to named entities, nouns, proper nouns, and numerical tokens extracted from the clue using a spaCy NLP pipeline, discarding function words, modifier terms, and stylistic connectives that mostly would not appear in the target article. We conclude that the entity mode works very well for clues

using the *this* ... construction, where function words such as *this*, *it*, *so*, and *its* would otherwise dilute the query signal.

The base query text is formed from the clue alone, or from the category and clue concatenated when the `--include-category` flag is enabled. Category inclusion is most beneficial for domain-specific categories that strongly constrain the answer type, such as media categories or geographic categories, but can introduce noise for generic categories.

For sparse retrieval, the transformed query string is submitted directly to the Whoosh index. A dedicated year-match component extracts all four-digit year tokens from the query and issues a conjunctive Boolean query requiring each extracted year to appear in the candidate article's title or body. This provides a targeted high-precision signal for the 41% of clues that contain explicit temporal anchors, and is normalized separately before being incorporated into the final score.

For dense retrieval, query embeddings are produced using the DPR question encoder. The default embedding is derived from the prompt *"The clue is: {clue}"* or, when the category is included, *"The category is: {category}. The clue is: {clue}"*. To address the lexical mismatch between Jeopardy! clue phrasing and Wikipedia article vocabulary, the system additionally uses a Qwen-3 4B language model (**?**) to reformulate each clue into five natural-language questions. For example, a clue such as *"This woman who won consecutive heptathlons at the Olympics went to UCLA on a basketball scholarship"* might be reformulated as *"Which athlete won back-to-back heptathlons at the Olympics and attended UCLA?"*, a phrasing that more closely resembles the language of the target Wikipedia article. Each of the five generated questions is encoded separately with the DPR question encoder; their retrieval scores are then averaged to produce a dense signal that is robust to any single reformulation. A sixth embedding is produced from the concatenation of all five questions into a single input string, providing a complementary dense query signal. All query embeddings are precomputed and cached to avoid repeated model inference during evaluation.

### 2.5 Score Normalization and Final Ranking

Because sparse BM25 scores and dense inner-product scores occupy incommensurable ranges, each retrieval component normalizes its scores before they are combined. Sparse components apply max normalization:

$$s_{\text{norm}} = \frac{s_{\text{raw}}}{\max_d s_{\text{raw}}(d)} \tag{1}$$

Dense FAISS components apply min-max normalization:

$$s_{\text{norm}} = \frac{s_{\text{raw}} - \min_d s_{\text{raw}}(d)}{\max_d s_{\text{raw}}(d) - \min_d s_{\text{raw}}(d)} \tag{2}$$

For the default FAISS component, which searches all four passage-granularity indexes, the normalized score assigned to a document is the maximum it receives across the four variants. For the natural-question average FAISS component, the per-document score is computed by first taking the maximum across the four FAISS variants for each of the five generated questions, and then averaging across the five question-level scores.

The final document score is a weighted linear combination of all normalized component scores:

$$\begin{aligned} \text{score}(d) \;=\; & w_{\text{body}} \cdot s_{\text{body}}(d) \;+\; w_{\text{faiss}} \cdot s_{\text{faiss}}(d) \\ & +\; w_{\text{nq-avg}} \cdot s_{\text{nq-avg}}(d) \;+\; w_{\text{nq-concat}} \cdot s_{\text{nq-concat}}(d) \\ & +\; w_{\text{year}} \cdot s_{\text{year}}(d) \end{aligned} \tag{3}$$

where the current non-zero weights are $w_{\text{body}} = 20$, $w_{\text{faiss}} = w_{\text{nq-avg}} = w_{\text{nq-concat}} = w_{\text{year}} = 1$. The body BM25 score receives the highest weight because lexical matching over the full article body is the most reliable signal for the majority of clues; the dense and year-match components provide complementary evidence

that improves recall on clues where lexical overlap between the clue and the correct article is limited. Each component retrieves up to 10,000 candidate articles before score merging, and component results are cached in SQLite using the query text, category, component limit, index fingerprints, and query-embedding digest as cache keys.

# 3 Measuring Performance

## 3.1 Evaluation Metric

We evaluate our system using two complementary retrieval metrics: Precision at 1 (P@1) and Mean Reciprocal Rank (MRR). P@1 measures the fraction of the 100 Jeopardy! questions for which the correct Wikipedia article appears as the top-ranked result, and is the primary metric for this task. It is the natural choice here because each question has exactly one correct answer and the system is expected to commit to a single prediction, mirroring the structure of the Jeopardy! game itself. Normalized Discounted Cumulative Gain (NDCG), while appropriate for ranked retrieval tasks with graded relevance, is less suitable here because there is no meaningful notion of partial relevance: an article is either the correct answer or it is not. MRR is reported as a secondary metric because it gives partial credit when the correct answer appears at rank 2 or 3, providing a softer signal that is useful for diagnosing whether errors are near misses or complete failures. Formally, for a set of $N$ questions:

$$\text{P@1} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\text{rank}_i = 1] \tag{4}$$

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{\text{rank}_i} \tag{5}$$

where $\text{rank}_i$ is the rank of the correct article for question $i$, and the sum in MRR is taken only over questions where the correct article appears in the top-$k$ returned results ($k = 10$ in our experiments).

## 3.2 Results

Table 1 presents P@1 and MRR across five system configurations in an ablation study that isolates the contribution of each retrieval component. The baseline system uses only BM25 body retrieval. Subsequent rows add the default DPR FAISS component, the natural-question average FAISS component, the concatenated natural-question FAISS component, and finally the full JEOPARDY_PROGRAM system which additionally incorporates the year-match bonus.

Table 2 compares the tuned weighted configuration against a uniform-weight baseline in which all active components are assigned equal weight ($w = 1.0$), isolating the effect of weight tuning from the effect of component selection.

## 3.3 Discussion

We draw several observations from these results. First, adding DPR dense retrieval on top of BM25 body retrieval improves both P@1 and MRR, confirming that dense and sparse signals are complementary: BM25 handles clues with strong lexical overlap between the clue and the target article, while DPR recovers articles where the clue and article use different vocabulary to express the same concept. Second, the natural-question augmentation components provide additional gains, with the averaged embeddings over five reformulations outperforming the single concatenated embedding, suggesting that averaging over diverse query phrasings

Table 1: Ablation results on the 100-question Jeopardy! benchmark. P@1 is the primary metric; MRR is reported as a secondary measure. Each row adds one retrieval component over the previous configuration. Best results per column are in **bold**.

| System Configuration | P@1 | MRR |
|---|---|---|
| Body BM25 (baseline) | *XX.X* | *XX.X* |
| + FAISS (DPR default) | *XX.X* | *XX.X* |
| + NQ avg FAISS | *XX.X* | *XX.X* |
| + NQ concat FAISS | *XX.X* | *XX.X* |
| + Year match (JEOPARDY_PROGRAM) | ***XX.X*** | ***XX.X*** |

Table 2: Comparison of the equal-weight configuration (`--weight-equal`, all active components set to $w = 1.0$) against the tuned JEOPARDY_PROGRAM configuration ($w_{\text{body}} = 20$, all other active components $w = 1$). Best results per column are in **bold**.

| Configuration | P@1 | MRR |
|---|---|---|
| Equal weights (`--weight-equal`) | *XX.X* | *XX.X* |
| Tuned weights (JEOPARDY_PROGRAM) | ***XX.X*** | ***XX.X*** |

produces a more robust dense signal than concatenation. Third, the year-match bonus provides a meaningful improvement on the full benchmark, consistent with the dataset analysis showing that 41 of the 100 clues contain explicit four-digit years that appear in the target article. Finally, Table 2 shows that assigning higher weight to the body BM25 component is beneficial: with equal weights, the dense components can overwhelm the sparse signal on clues where lexical matching is in fact the stronger retrieval mechanism, whereas the tuned configuration preserves the primacy of BM25 while still benefiting from the complementary dense evidence.